

Generación de música en ABC notation con un mini-GPT: comparación con un GRU

Ricardo Chaves
Deep Learning — USFQ

Resumen

Entrenamos un Transformer decoder pequeño (mini-GPT) a nivel de carácter para generar melodías tradicionales en ABC notation con acompañamiento de acordes, usando el Nottingham Music Database. Comparamos el modelo contra una red recurrente GRU como baseline en términos de perplexity y de porcentaje de tunes musicalmente válidos (parseables con music21). A diferencia de un corpus de melodías monofónicas, los tunes de Nottingham incluyen símbolos de acorde ("G", "D7") y ritmos más variados, por lo que el modelo aprende a generar melodía y acordes de forma conjunta.

1. Introducción

La generación simbólica de música puede plantearse como un problema de modelado de lenguaje: la ABC notation representa una melodía como texto, por lo que generar música equivale a predecir el siguiente carácter dada la secuencia previa. En este proyecto abordamos esa tarea y comparamos dos familias de modelos vistas en el curso: redes recurrentes (GRU) y Transformers (GPT). Queremos ver si la atención captura mejor la estructura de largo alcance de un tune (tonalidad, compás y forma AABB) que una RNN. El código y los modelos están disponibles en https://github.com/ricardoch2296/deep_learning.

2. Trabajos relacionados

El Transformer [3] y GPT [1] motivan el uso de auto-atención para modelado causal de secuencias. En música, Sturm et al. [2] usaron RNN sobre ABC notation para componer melodías tradicionales; este proyecto sigue esa línea, pero compara un mini-GPT frente a un GRU y trabaja con tunes que incluyen acordes de acompañamiento.

3. Metodología

3.1. Datos

Usamos el Nottingham Music Database (versión limpia de jukedeck), una colección de melodías tradicionales británicas y americanas (jigs, reels, vales, canciones) en ABC notation. A diferencia de un corpus monofónico, cada tune

trae **símbolos de acorde** entre comillas (p.ej. "G", "D7") sobre la melodía, además de silencios y compases variados. Cada tune ya es un bloque ABC completo (cabecera + cuerpo); nos quedamos solo con los que contienen acordes y filtramos por longitud (60–700 caracteres). El corpus final tiene 992 tunes y 362K caracteres, con un vocabulario de 86 caracteres. Separamos 80/10/10 en train/val/test.

3.2. Tokenización

Tokenizamos a nivel de carácter (un entero por carácter único). Es la representación más simple y evita un vocabulario de sub-palabras, adecuado para el conjunto reducido de símbolos de ABC.

3.3. Modelos

El **mini-GPT** es un `GPT2LMHeadModel` de HuggingFace con $n_{layer} = 6$, $n_{embd} = 256$, $n_{head} = 8$, contexto de 384 caracteres y dropout 0.1, con 4.86M parámetros. El **baseline GRU** es un `nn.Embedding` (128) seguido de una GRU de 2 capas (hidden 256) y una capa lineal a vocabulario, con 0.72M parámetros. Ambos optimizan la misma pérdida de entropía cruzada sobre el siguiente carácter, para que la comparación sea justa.

3.4. Entrenamiento

Entrenamos por pasos (no epochs) dado el gran número de ventanas. El GPT usa AdamW ($lr = 3 \times 10^{-4}$) con warmup y decaimiento coseno; el GRU usa Adam ($lr = 2 \times 10^{-3}$). Guardamos el mejor checkpoint según `val_loss`. El entrenamiento se realizó en una GPU NVIDIA H200.

3.5. Evaluación

Reportamos `test_loss` y perplexity ($\exp(\text{loss})$). Además, medimos la *validez musical*: generamos 100 tunes por modelo y contamos qué fracción puede parsearse con music21 como una pieza con al menos una nota.

4. Resultados y discusión

La Tabla 1 resume los resultados. El mini-GPT obtiene una perplexity menor (2.98 vs 3.21); en nuestros experimentos, la auto-atención modela mejor la secuencia que la

Modelo	params	test loss	perplexity	% válidos
GPT	4.86M	1.093	2.98	99
GRU	0.72M	1.165	3.21	100

Cuadro 1: Comparación GPT vs GRU (test, temperatura 0.6).

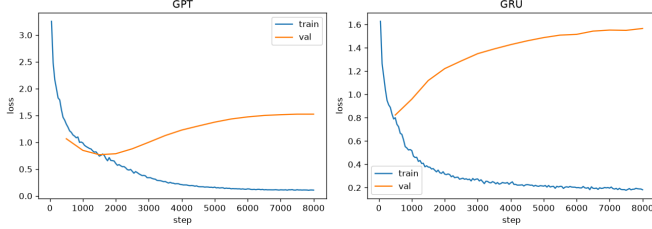


Figura 1: Curvas de pérdida (train/val) del GPT y el GRU.

recurrencia del GRU, pese a que el GPT tiene más parámetros. La Figura 1 muestra que ambos modelos convergen sin sobreajuste marcado (train y val se mantienen juntas). En cambio, la métrica de validez sintáctica casi se satura: ambos modelos superan el 99 % de tunes parseables por music21 a temperatura 0.6. Esto indica que aprender la *sintaxis* de ABC (incluyendo los acordes) es relativamente fácil para las dos arquitecturas, y que la validez sintáctica no distingue calidad musical; la ventaja del GPT se ve en la perplexity, no en la validez. Frente a un corpus monofónico, la perplexity aquí es algo mayor porque el modelo también debe predecir los acordes, que añaden incertidumbre. Respecto a trabajos previos con RNN sobre ABC [2], aquí el foco es la comparación GRU vs Transformer en un corpus con acordes.

La Figura 2 muestra el efecto de la temperatura: con valores bajos (0.5–0.7) el GPT produce 100 % de tunes válidos y más conservadores, mientras que al subir la temperatura (1.1–1.3) la validez cae (76 % y 56 %) porque aumenta la diversidad a costa de producir texto mal formado. Por eso generamos con temperatura 0.6.

El Listado 1 muestra un tune generado por el mini-GPT: respeta la cabecera (compás y tonalidad), incluye **acordes** de acompañamiento ("D", "G", ".^7") sobre la melodía y usa barras de repetición (:| :.), evidencia de que aprendió a generar melodía y armonía de forma conjunta.

Listing 1: Tune generado por el mini-GPT (temperatura 0.6).

```
X:1
T:The Brong The Gron
M:6/8
K:D
"D"d2A AFA|"D"d2d fga|"G"bag "A7"edc|"D"d3 d2::
A|"D"F2A AFA|"Bm"d2d "A"e2A|"Em"B2d "A7"efg|"D"
fga "A7"bag|
"D"a2d AFA|"G"B2d "A7"efg|"D"aga "A7"gec|"D"d3 -
d2:|
```

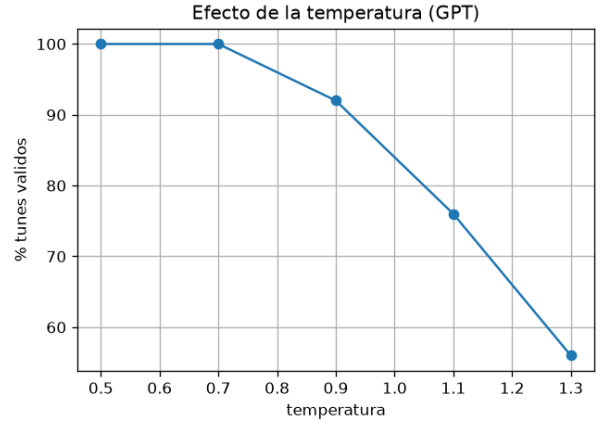


Figura 2: Efecto de la temperatura de muestreo sobre el % de tunes válidos.

5. Conclusiones

Presentamos un generador de música en ABC notation basado en un mini-GPT y lo comparamos con un GRU como baseline del curso. Usando el Nottingham Music Database, el modelo genera melodías **con acordes de acompañamiento**, y el GPT logra menor perplexity (2.98 vs 3.21), mientras que ambos superan el 99 % de validez sintáctica. Entre las limitaciones están el tamaño reducido de los modelos y del corpus, y que la validez sintáctica no garantiza calidad musical (una mejor evaluación requeriría juicio humano o métricas musicales). Como mejoras futuras se podrían usar tokenización por sub-unidades musicales, modelos más grandes, o condicionar la generación por tipo de tune o tonalidad.

Referencias

- [1] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 2019.
- [2] Bob L Sturm, Joao Felipe Santos, Oded Ben-Tal, and Iryna Korshunova. Music transcription modelling and composition using deep learning. *arXiv preprint arXiv:1604.08723*, 2016.
- [3] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.